

Emporix Punchout Plugin — Complete Guide

Emporix Punchout Plugin — Complete Guide

Version: 1.0.0

Date: 2026-05-12

Status: Production Ready

Table of Contents

1. [Overview](#)
 2. [Architecture](#)
 3. [Protocol Flows](#)
 4. [Installation & Deployment](#)
 5. [Registering the Extension in Emporix](#)
 6. [Admin UI Configuration](#)
 7. [Storefront Integration](#)
 8. [Security Model](#)
 9. [API Reference](#)
 10. [Buyer Configuration \(cXML\)](#)
 11. [Buyer Configuration \(OCI\)](#)
 12. [Troubleshooting](#)
 13. [Environment Variables](#)
 14. [Data Model Reference](#)
-

Overview

The Emporix Punchout Plugin transforms an Emporix-powered supplier store into a punchout-enabled catalog. Enterprise buyers authenticate from their procurement systems, shop the Emporix storefront, and return a filled cart to their procurement system — without creating an Emporix order.

Supported protocols at launch:

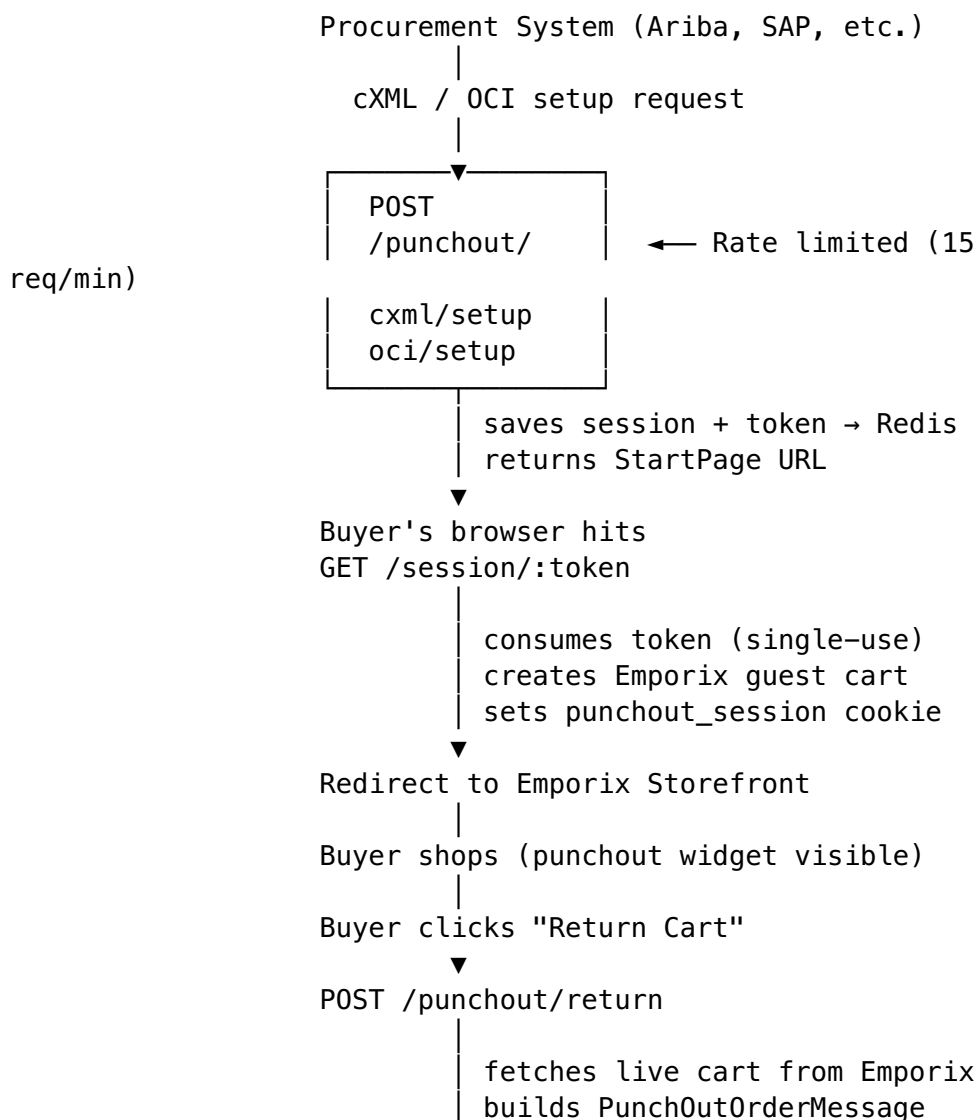
Protocol	Version	Procurement Systems
cXML	1.2	Ariba, Coupa, Jaggaer, Workday
OCI	4.0	SAP SRM, SAP S/4HANA, Oracle

Key capabilities:

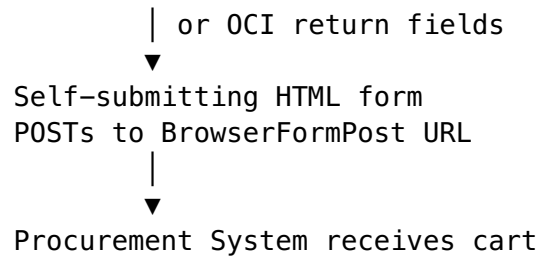
- Standards-compliant cXML PunchOutSetupRequest / PunchOutOrderMessage flow
- OCI form-based setup and return
- Buyer org → customer group mapping for segmented pricing
- Redis-backed session store with replay-attack prevention
- Merchant admin UI embedded in the Emporix Management Dashboard
- Storefront “Return Cart” widget injected via JS snippet

Architecture

Single Express/TypeScript service deployed as a Docker container. Stateless beyond a Redis session store.



(cXML)



Internal modules

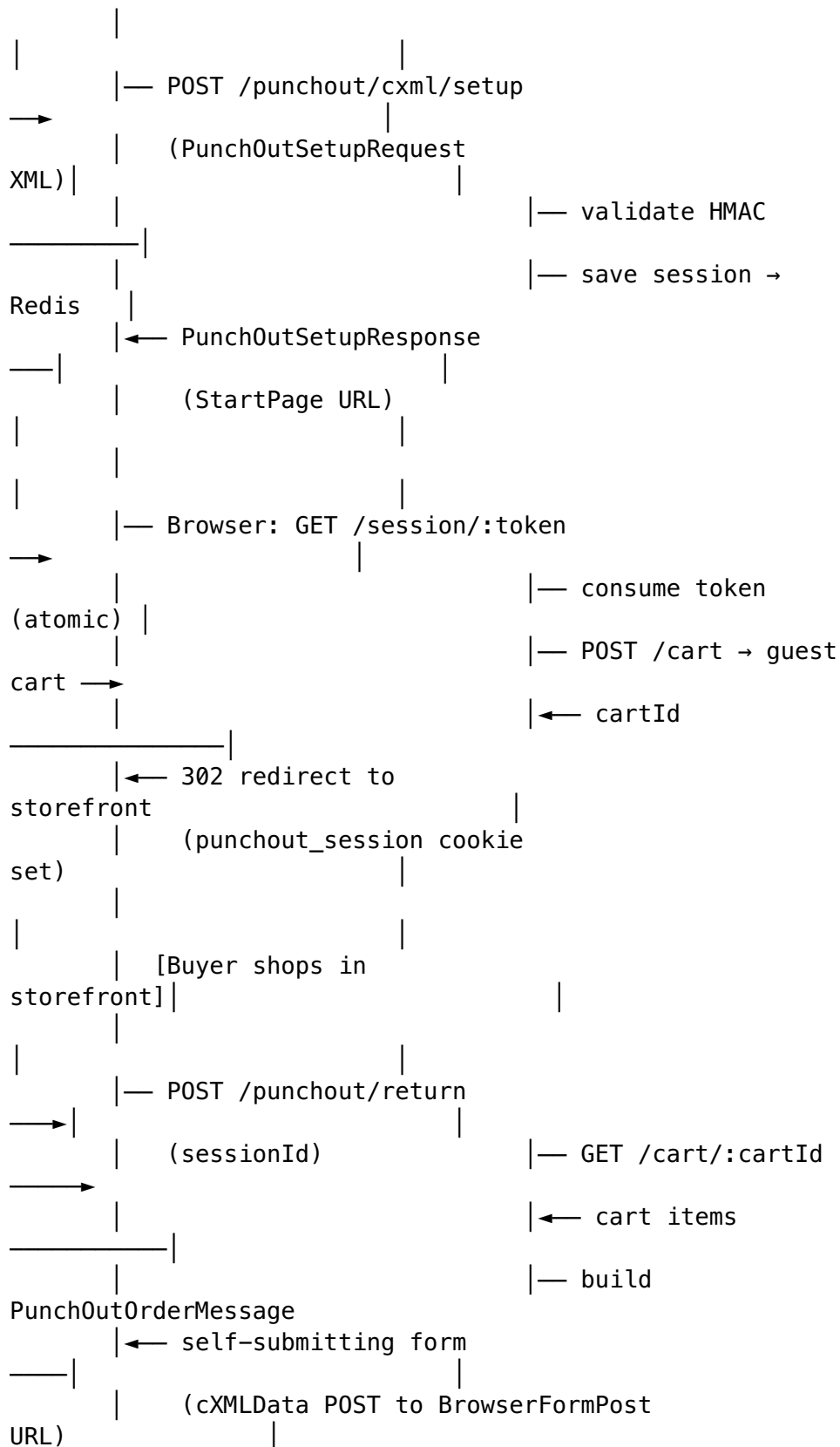
Module	Path	Responsibility
Protocol — cXML	src/protocols/ cxml/	Parse PunchOutSetupRequest, generate PunchOutSetupResponse and PunchOutOrderMessage
Protocol — OCI	src/protocols/ oci/	Parse OCI form fields, generate indexed OCI return fields
Session store	src/session/	Redis CRUD for one- time tokens and session records
Emporix client	src/emporix/	OAuth2 token cache, cart API, customer group API
Admin config store	src/admin/ configStore.ts	Read/write plugin config via Emporix Configuration Service
Admin router	src/admin/ router.ts	JWT-protected /admin/* routes
Middleware	src/middleware/	Rate limiter, Emporix JWT verification
Routes	src/routes/	Punchout setup/return, session start, punchout widget JS
Admin UI	admin-ui/	React + Vite module federation extension for Emporix dashboard

Protocol Flows

cXML Punchout Flow

Procurement System
Emporix

Plugin



Step-by-step:

1. Procurement system sends PunchOutSetupRequest containing buyer org identity, shared secret, BuyerCookie, and BrowserFormPost return URL
2. Plugin validates shared secret (bcrypt compare), resolves buyer org → customer group via buyer mapping table

3. Plugin creates session in Redis with 15-minute one-time token + 2-hour session record
4. Plugin returns PunchOutSetupResponse with StartPage URL
5. Browser hits /session/:token — token atomically consumed and deleted (prevents replay), Emporix guest cart created scoped to customer group, session cookie set, redirect to storefront
6. Buyer shops in Emporix storefront (punchout widget button visible)
7. Buyer clicks “Return Cart” — browser POSTs sessionId to /punchout/return
8. Plugin fetches live cart from Emporix, translates each line item to a cXML ItemIn, wraps in PunchOutOrderMessage
9. Plugin returns a self-submitting HTML form that POSTs the XML to the BrowserFormPost URL via the buyer’s browser
10. Procurement system receives the PunchOutOrderMessage and populates the requisition

OCI Punchout Flow

Same flow with protocol-specific differences:

1. Setup arrives as form POST with fields HOOK_URL, USERNAME, PASSWORD, ~OkCode, ~Caller
2. Plugin validates PASSWORD against stored shared secret
3. Plugin returns HTML redirect (not XML) to session start URL
4. Return posts OCI-formatted fields to saved HOOK_URL

OCI field mapping on return:

OCI Field	Source
NEW_ITEM-DESCRIPTION[n]	Cart item name
NEW_ITEM-MATNR[n]	Cart item SKU
NEW_ITEM-QUANTITY[n]	Cart item quantity
NEW_ITEM-UNIT[n]	Cart item unit of measure
NEW_ITEM-PRICE[n]	Cart item unit price
NEW_ITEM-CURRENCY[n]	Cart currency
NEW_ITEM-VENDMAT[n]	Vendor material number (SKU)
~OkCode	Configured value (default: ADDFROMCATALOG)

Session Expiry Handling

Two-tier TTL strategy:

TTL	Key	Purpose
15 minutes	punchout:session:<token>	One-time token — deleted on first use
2 hours	punchout:active:<sessionId>	

TTL	Key	Purpose
		Full session record

Token expired before storefront load: `/session/:token` returns a user-facing HTML error page.

Session lost during shopping: On return request, if session lookup fails, plugin returns a protocol-appropriate error page. Buyer must restart punchout.

Installation & Deployment

Prerequisites

- Docker and Docker Compose
- A publicly reachable HTTPS domain for the plugin service
- Emporix tenant with service account credentials
- Redis (included in docker-compose)

Step 1 — Configure environment

```
cp .env.example .env
```

Edit `.env`:

Variable	Value
PLUGIN_HOST	Public HTTPS URL (e.g. <code>https://punchout.your-domain.com</code>)
EMPORIX_TENANT_ID	Your Emporix tenant ID
EMPORIX_CLIENT_ID	Emporix service account client ID
EMPORIX_CLIENT_SECRET	Emporix service account client secret
AES_KEY	Exactly 32 random bytes (see below)

Generate a secure AES key:

```
openssl rand -base64 24 | tr -d '=' | cut -c1-32
```

Step 2 — Deploy

```
docker-compose up -d
```

Verify the service is running:

```
curl https://your-plugin-domain.com/health
# → {"status":"ok"}
```

Step 3 — Register the extension

See [Registering the Extension in Emporix](#) below.

Running locally (development)

```
npm install
cp .env.example .env    # fill in values
redis-server &          # start local Redis
npm run build
node dist/index.js
```

For the admin UI in dev mode:

```
cd admin-ui
npm install
# .env.local already has VITE_PLUGIN_HOST=http://
# localhost:3000
npm run dev
# visit http://localhost:5173/
# add #token=<your-emporix-jwt> to the URL for authenticated
# calls
```

Registering the Extension in Emporix

The admin UI is a **module federation** extension loaded by the Emporix Management Dashboard. It does **not** use a `plugin.json` manifest.

Registration steps:

1. Log into the Emporix Management Dashboard
2. Navigate to **Administration** → **Extensions**
3. Click **Add extension**
4. Provide:
 - **Name:** Punchout Connector (appears in dashboard navigation)
 - **Module toggle:** ON
 - **Module URL:** `https://your-plugin-domain.com/admin-ui/assets/remoteEntry.js`
5. Save

The Emporix dashboard will load `remoteEntry.js` and render the punchout admin UI inside an `iframe/panel`. The dashboard automatically passes `appState: { tenant, token, language }` as props — no additional authentication setup required.

Admin UI Configuration

Once registered, access the extension from the Emporix Management Dashboard. Four configuration screens are available:

Credentials

Configure the shared secret and Emporix service account:

- **Shared Secret** — the secret that procurement systems use to authenticate. Leave blank to keep the existing value. Stored bcrypt-hashed; never returned via API.
- **Emporix Service Account Client ID** — used by the plugin to create guest carts on behalf of buyers
- **Emporix Service Account Client Secret** — stored AES-256-GCM encrypted at rest
- **Storefront Base URL** — the URL of your Emporix storefront (e.g. `https://shop.your-domain.com`)

Buyer Mappings

Map buyer org IDs from procurement systems to Emporix customer groups for segmented pricing:

- **Buyer Org ID** — the identity from cXML BuyerCookie or OCI USERNAME
- **Customer Group** — dropdown populated from Emporix customer groups API

If no mapping exists for a buyer, the default Emporix customer group is used (standard catalog prices).

Protocol Settings

- **Enable cXML / Enable OCI** — toggle protocol support
- **cXML operationAllowed** — edit (buyers may modify requisition after return), create, or inspect
- **OCI ~OkCode** — ADDFROMCATALOG (default) or SOURCING

Connection Test

Sends a simulated cXML PunchOutSetupRequest to the plugin and displays the raw HTTP response. Replace YOUR_SHARED_SECRET in the payload with your configured shared secret before clicking **Run Connection Test**.

Storefront Integration

Add the punchout widget to your Emporix storefront's <head>:

```
<script src="https://your-plugin-domain.com/punchout-  
widget.js"></script>
```

The widget automatically: - Detects active punchout sessions via the punchout_session cookie - Renders a “Return Cart to Procurement System” button (fixed position, bottom-right) - On click: submits the session ID to /punchout/return

The widget does nothing for non-punchout visitors — it exits silently if no session cookie is present.

Security Model

Credential storage

Secret	Storage method
Shared secret	bcrypt hash (10 rounds), never returned via API
Service account client secret	AES-256-GCM encrypted, IV + auth tag stored alongside
AES encryption key	Environment variable only, never persisted

Request validation

- **cXML:** Shared secret validated via bcrypt compare on every setup request
- **OCI:** Password validated via bcrypt compare on every setup request
- **Admin routes:** Emporix JWT verified via JWKS endpoint (RS256), tenant ID extracted from token claims
- **All payloads:** XML validated with fast-xml-parser XMLValidator before parsing; malformed input never surfaces raw errors

Session security

- One-time tokens are atomically consumed (Redis MULTI/GET/DEL pipeline) — replay attacks prevented
- Session cookies: httpOnly: true, sameSite: lax, secure: true in production
- Sessions auto-expire after 2 hours

Rate limiting

Punchout endpoints (cXML setup, OCI setup) are rate-limited to **15 requests per minute per source IP**.

Output encoding

- cXML error text: XML-escaped (&, <, >, ", ')
- OCI field values: HTML-escaped in self-submitting form
- hookUrl / browserFormPostUrl: HTML-escaped in form action attribute

Headers

- All responses: helmet security headers (XSS protection, MIME sniffing prevention, etc.)
 - Admin routes: Content-Security-Policy: default-src 'self'; frame-ancestors 'self' https://admin.emporix.io
 - Admin UI static assets: CORS restricted to https://admin.emporix.io
-

API Reference

Public punchout endpoints

No Emporix authentication required. Called directly by procurement systems or buyer browsers.

POST /punchout/cxml/setup

Accepts a cXML 1.2 PunchOutSetupRequest. Returns PunchOutSetupResponse with StartPage URL.

Request:

Content-Type: text/xml

```
<?xml version="1.0" encoding="UTF-8"?>
<cXML payloadID="..." timestamp="...">
  <Header>
    <From><Credential
domain="NetworkId"><Identity>{buyerOrgId}</Identity></
Credential></From>
    <To><Credential
domain="NetworkId"><Identity>{supplierId}</Identity></
Credential></To>
    <Sender>
      <Credential domain="NetworkId">
        <Identity>{buyerOrgId}</Identity>
```

```

        <SharedSecret>{sharedSecret}</SharedSecret>
    </Credential>
    <UserAgent>{systemName}</UserAgent>
</Sender>
</Header>
<Request deploymentMode="production">
    <PunchOutSetupRequest operation="create">
        <BuyerCookie>{buyerCookie}</BuyerCookie>
        <BrowserFormPost><URL>{returnUrl}</URL></
BrowserFormPost>
    </PunchOutSetupRequest>
</Request>
</cXML>

```

Response (200): PunchOutSetupResponse XML with StartPage URL

Response (401): Invalid shared secret

Response (503): Plugin not configured

POST /punchout/oci/setup

Accepts OCI 4.0 form fields. Returns HTTP 302 redirect to session start URL.

Request:

Content-Type: application/x-www-form-urlencoded

HOOK_URL=https://sap.example.com/oci/return

USERNAME=buyer-org-id

PASSWORD=shared-secret

~OkCode=ADDFROMCATALOG

~Caller=SAPLMOB

Response (302): Redirect to /session/:token

GET /session/:token

Validates one-time token, creates Emporix guest cart, sets session cookie, redirects to storefront.

Response (302): Redirect to {storefrontBaseUrl}?cartId={cartId}

Response (410): Token expired or already used

Response (503): Plugin not configured

POST /punchout/return

Reads active session, fetches live cart, returns self-submitting form that delivers cart to procurement system.

Request body:

Content-Type: application/x-www-form-urlencoded

sessionId={sessionId}

Response (200): Self-submitting HTML form (browser auto-submits to procurement system)

Response (410): Session expired

Admin endpoints

All require Authorization: Bearer {emporixJwt} header. Called by the admin UI.

Method	Path	Description
GET	/admin/config	Read plugin config (secrets masked)
POST	/admin/config	Save plugin config
GET	/admin/buyers	List buyer org → customer group mappings
POST	/admin/buyers	Add or update a buyer mapping
GET	/admin/customer-groups	List Emporix customer groups (for dropdown)

Static assets

Path	Description
GET /punchout-widget.js	Storefront punchout return button snippet
GET /admin-ui/assets/remoteEntry.js	Module federation entry point for Emporix dashboard
GET /health	Health check — returns {"status":"ok"}

Buyer Configuration (cXML)**What to give buyers (Ariba, Coupa, Jaggaer, Workday)**

Buyers configure their procurement system with:

Field	Value
Punchout URL	https://your-plugin-domain.com/punchout/cxml/setup
Shared Secret	The value you set in Admin UI → Credentials
Supplier Identity	Your supplier ID (any value; not validated)

Buyer org mapping

In Admin UI → Buyer Mappings, add the buyer's org identity (the value in their cXML <From><Credential><Identity>) and map it to the appropriate Emporix customer group.

Buyer Configuration (OCI)

What to give buyers (SAP SRM / S/4HANA)

Buyers configure a punchout catalog in their SAP system with:

SAP Field	Value
URL	https://your-plugin-domain.com/punchout/oci/setup
Username	The buyer's org ID (used for customer group mapping)
Password	The shared secret set in Admin UI → Credentials
~OkCode	ADDFROMCATALOG

Troubleshooting

“Your punchout session has expired”

The buyer's browser hit /session/:token more than 15 minutes after setup, or the token was already consumed. The buyer must return to their procurement system and initiate punchout again.

401 on punchout setup

The shared secret in the procurement system doesn't match what's configured in Admin UI → Credentials.

503 on punchout setup

The plugin is not configured yet. Complete the Credentials screen in the Admin UI first.

Admin UI not loading in Emporix dashboard

1. Verify the plugin service is running and HTTPS is working: `curl https://your-domain.com/health`
2. Verify `remoteEntry.js` is accessible: `curl https://your-domain.com/admin-ui/assets/remoteEntry.js`
3. Check the registered URL in Administration → Extensions matches the actual URL
4. CORS is restricted to `https://admin.emporix.io` — the extension will not load from other origins

Cart not returning to procurement system

1. Check the `BrowserFormPost` URL in the cXML setup request — it must be reachable from the buyer's browser
 2. Check the session hasn't expired (2-hour window)
 3. Verify the Emporix cart API is responding — check plugin logs for 502 errors
-

Environment Variables

Variable	Required	Description
PORT	No	HTTP port (default: 3000)
PLUGIN_HOST	Yes	Public HTTPS URL of this service
REDIS_HOST	No	Redis hostname (default: localhost)
REDIS_PORT	No	Redis port (default: 6379)
EMPORIX_API_BASE	No	Emporix API base URL (default: <code>https://api.emporix.io</code>)
EMPORIX_TENANT_ID	Yes	Emporix tenant ID
EMPORIX_JWKS_URI	No	Emporix JWKS endpoint for JWT verification
EMPORIX_CLIENT_ID	Yes	Bootstrap service account client ID
EMPORIX_CLIENT_SECRET	Yes	Bootstrap service account client secret

Variable	Required	Description
AES_KEY	Yes	Exactly 32 bytes — used to encrypt stored credentials
OUTBOUND_TIMEOUT_MS	No	HTTP call timeout in ms (default: 5000)

Data Model Reference

Redis key schema

punchout:session:<token> # one-time token → sessionId
 (15 min TTL, deleted on use)
 punchout:active:<sessionId> # full session record (2 hr
 TTL)

Session record

```
{
  sessionId: string           // UUID
  protocol: 'cxml' | 'oci'
  buyerOrgId: string
  customerGroupId:
    string // empty string if no mapping found →
    Emporix default group
  browserFormPostUrl:
    string // BrowserFormPost URL (cXML) or HOOK_URL
    (OCI)
  buyerCookie: string        // cXML BuyerCookie; empty
    for OCI
  emporixCartId: string | null // set after session start
  createdAt: string           // ISO 8601
}
```

Plugin configuration (stored in Emporix Configuration Service)

```
{
  sharedSecretHash: string    // bcrypt hash – never
    returned via API
  serviceAccount: {
    clientId: string
    clientSecret: string      // AES-256-GCM
    encrypted at rest
  }
  storefrontBaseUrl: string
  cxmlEnabled: boolean
  ociEnabled: boolean
}
```

```
operationAllowed: 'create' | 'edit' | 'inspect'  
ociOkCode: 'ADDFROMCATALOG' | 'SOURCING'  
buyerMappings: Array<{  
  buyerOrgId: string  
  customerGroupId: string  
}
```